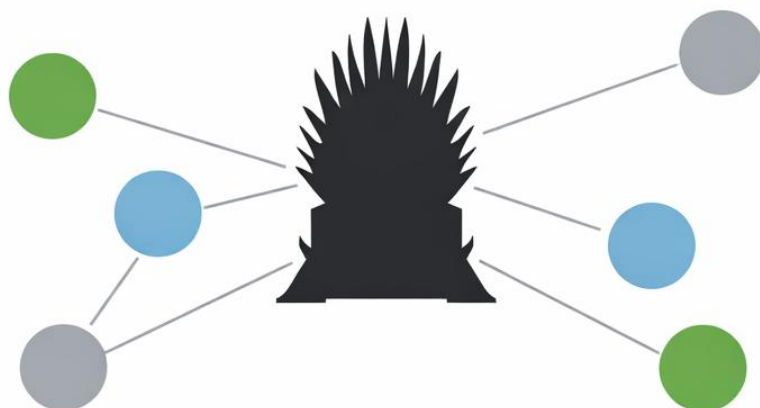


PRUEBA CON

Neo4j

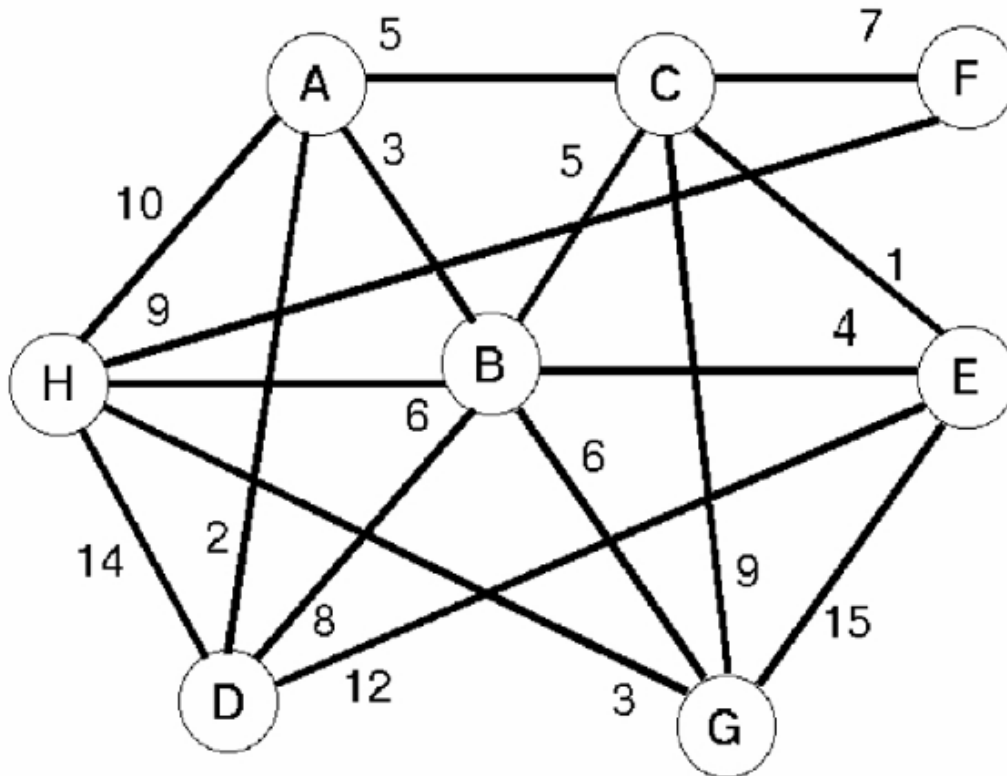
(JUEGO DE TRONOS)



Ivana Sánchez Pérez

PRUEBA CON Neo4J (JUEGO DE TRONOS)

Ejercicio 1. Análisis de Grafo Manual



a) Creación del Grafo

a) Creación del Grafo Se ha procedido a la creación del grafo en Neo4j siguiendo el esquema proporcionado. Se han definido nodos con la etiqueta: elemento y relaciones con la etiqueta: camino.

- **Propiedades:** Cada nodo incluye la propiedad nombre (A-H) y cada relación la propiedad distancia con los pesos correspondientes (ej. F-C: 7, G-D: 12).
- **Implementación:** Se han creado relaciones bidireccionales de forma explícita para asegurar la navegabilidad total del grafo.

neo4j\$

To help make Neo4j Browser better we collect information on product usage. Review your [settings](#) at any time.

neo4j\$ // Crear Nodos CREATE (a:elemento {nombre: 'A'}), (b:elemento {nombre: 'B'}), (c:elemento ...

Server version	Neo4j/4.4.15
Server address	localhost:7687
Query	// Crear Nodos CREATE (a:elemento {nombre: 'A'}), (b:elemento {nombre: 'B'}), (c:elemento {nombre: 'C'}), (d:elemento {nombre: 'D'}), (e:elemento {nombre: 'E'}), (f:elemento {nombre: 'F'}), (g:elemento {nombre: 'G'}), (h:elemento {nombre: 'H'})
Summary	{ "query": { "text": "// Crear Nodos\n\nCREATE (a:elemento {nombre: 'A'}), (b:elemento {nombre: 'B'}), (c:elemento {nombre: 'C'}), \n\n(d:elemento {nombre: 'D'}), (e:elemento {nombre: 'E'}), (f:elemento {nombre: 'F'}), \n\n(g:elemento {nombre: 'G'}), (h:elemento {nombre: 'H'})", ...
Response	[] ...

Added 8 labels, created 8 nodes, set 8 properties, completed after 506 ms.

neo4j\$

To help make Neo4j Browser better we collect information on product usage. Review your [settings](#) at any time.

neo4j\$ // Crear Relaciones (Bidireccionales mediante dos flechas o lógica de grafo no dirigido) M...

Server version	Neo4j/4.4.15
Server address	localhost:7687
Query	// Crear Relaciones (Bidireccionales mediante dos flechas o lógica de grafo no dirigido) MATCH (a:elemento {nombre: 'A'}), (b:elemento {nombre: 'B'}), (c:elemento {nombre: 'C'}), (d:elemento {nombre: 'D'}), (e:elemento {nombre: 'E'}), (f:elemento {nombre: 'F'}), (g:elemento {nombre: 'G'}), (h:elemento {nombre: 'H'}) CREATE (a)-[:camino {distancia: 5}]->(c), (c)-[:camino {distancia: 5}]->(a), (a)-[:camino {distancia: 3}]->(b), (b)-[:camino {distancia: 3}]->(a), (a)-[:camino {distancia: 10}]->(h), (h)-[:camino {distancia: 10}]->(a), (a)-[:camino {distancia: 2}]->(d), (d)-[:camino {distancia: 2}]->(a), (c)-[:camino {distancia: 7}]->(f), (f)-[:camino {distancia: 7}]->(c), (c)-[:camino {distancia: 5}]->(b), (b)-[:camino {distancia: 5}]->(c), (c)-[:camino {distancia: 9}]->(g), (g)-[:camino {distancia: 9}]->(c), (b)-[:camino {distancia: 6}]->(h), (h)-[:camino {distancia: 6}]->(b), (b)-[:camino {distancia: 4}]->(e), (e)-[:camino {distancia: 4}]->(b), (b)-[:camino {distancia: 6}]->(g), (g)-[:camino {distancia: 6}]->(b), (b)-[:camino {distancia: 8}]->(d), (d)-[:camino {distancia: 8}]->(b), (h)-[:camino {distancia: 9}]->(f), (f)-[:camino {distancia: 9}]->(h), (h)-[:camino {distancia: 14}]->(d), (d)-[:camino {distancia: 14}]->(h), (f)-[:camino {distancia: 1}]->(e), (e)-[:camino {distancia: 1}]->(f), (e)-[:camino {distancia: 15}]->(g), (g)-[:camino {distancia: 15}]->(e), (d)-[:camino {distancia: 12}]->(g), (g)-[:camino {distancia: 12}]->(d), (h)-[:camino {distancia: 3}]->(g), (g)-[:camino {distancia: 3}]->(h) { "query": { "text": "// Crear Relaciones (Bidireccionales mediante dos flechas o lógica de grafo no dirigido)\n\nMATCH (a:elemento {nombre: 'A'}), (b:elemento {nombre: 'B'}), (c:elemento {nombre: 'C'}), \n\n(d:elemento {nombre: 'D'}), (e:elemento {nombre: 'E'}), (f:elemento {nombre: 'F'}), \n\n(g:elemento {nombre: 'G'}), (h:elemento {nombre: 'H'})\n\nCREATE (a)-[:camino {distancia: 5}]->(c), (c)-[:camino {distancia: 5}]->(a), \n\n(a)-[:camino {distancia: 3}]->(b), (b)-[:camino {distancia: 3}]->(a), \n\n(a)-[:camino {distancia: 10}]->(h), (h)-[:camino {distancia: 10}]->(a), \n\n(a)-[:camino {distancia: 2}]->(d), (d)-[:camino {distancia: 2}]->(a), \n\n(c)-[:camino {distancia: 7}]->(f), (f)-[:camino {distancia: 7}]->(c), \n\n(c)-[:camino {distancia: 5}]->(b), (b)-[:camino {distancia: 5}]->(c), \n\n(c)-[:camino {distancia: 9}]->(g), (g)-[:camino {distancia: 9}]->(c), \n\n(b)-[:camino {distancia: 6}]->(h), (h)-[:camino {distancia: 6}]->(b), \n\n(b)-[:camino {distancia: 4}]->(e), (e)-[:camino {distancia: 4}]->(b), \n\n(b)-[:camino {distancia: 6}]->(g), (g)-[:camino {distancia: 6}]->(b), \n\n(b)-[:camino {distancia: 8}]->(d), (d)-[:camino {distancia: 8}]->(b), \n\n(h)-[:camino {distancia: 9}]->(f), (f)-[:camino {distancia: 9}]->(h), \n\n(h)-[:camino {distancia: 14}]->(d), (d)-[:camino {distancia: 14}]->(h), \n\n(f)-[:camino {distancia: 1}]->(e), (e)-[:camino {distancia: 1}]->(f), \n\n(e)-[:camino {distancia: 15}]->(g), (g)-[:camino {distancia: 15}]->(e), \n\n(d)-[:camino {distancia: 12}]->(g), (g)-[:camino {distancia: 12}]->(d), \n\n(h)-[:camino {distancia: 3}]->(g), (g)-[:camino {distancia: 3}]->(h)
Summary	{ "query": { "text": "// Crear Relaciones (Bidireccionales mediante dos flechas o lógica de grafo no dirigido)\n\nMATCH (a:elemento {nombre: 'A'}), (b:elemento {nombre: 'B'}), (c:elemento {nombre: 'C'}), \n\n(d:elemento {nombre: 'D'}), (e:elemento {nombre: 'E'}), (f:elemento {nombre: 'F'}), \n\n(g:elemento {nombre: 'G'}), (h:elemento {nombre: 'H'})\n\nCREATE (a)-[:camino {distancia: 5}]->(c), (c)-[:camino {distancia: 5}]->(a), \n\n(a)-[:camino {distancia: 3}]->(b), (b)-[:camino {distancia: 3}]->(a), \n\n(a)-[:camino {distancia: 10}]->(h), (h)-[:camino {distancia: 10}]->(a), \n\n(a)-[:camino {distancia: 2}]->(d), (d)-[:camino {distancia: 2}]->(a), \n\n(c)-[:camino {distancia: 7}]->(f), (f)-[:camino {distancia: 7}]->(c), \n\n(c)-[:camino {distancia: 5}]->(b), (b)-[:camino {distancia: 5}]->(c), \n\n(c)-[:camino {distancia: 9}]->(g), (g)-[:camino {distancia: 9}]->(c), \n\n(b)-[:camino {distancia: 6}]->(h), (h)-[:camino {distancia: 6}]->(b), \n\n(b)-[:camino {distancia: 4}]->(e), (e)-[:camino {distancia: 4}]->(b), \n\n(b)-[:camino {distancia: 6}]->(g), (g)-[:camino {distancia: 6}]->(b), \n\n(b)-[:camino {distancia: 8}]->(d), (d)-[:camino {distancia: 8}]->(b), \n\n(h)-[:camino {distancia: 9}]->(f), (f)-[:camino {distancia: 9}]->(h), \n\n(h)-[:camino {distancia: 14}]->(d), (d)-[:camino {distancia: 14}]->(h), \n\n(f)-[:camino {distancia: 1}]->(e), (e)-[:camino {distancia: 1}]->(f), \n\n(e)-[:camino {distancia: 15}]->(g), (g)-[:camino {distancia: 15}]->(e), \n\n(d)-[:camino {distancia: 12}]->(g), (g)-[:camino {distancia: 12}]->(d), \n\n(h)-[:camino {distancia: 3}]->(g), (g)-[:camino {distancia: 3}]->(h)

Set 34 properties, created 34 relationships, completed after 169 ms.

b) Recorrido en Anchura (BFS) desde el nodo H

neo4j\$ CALL gds.graph.project('grafoManual', 'elemento', 'camino');

nodeProjection	relationshipProjection	graphName	nodeCount	relationshipCount	projectMillis
1	{ "elemento": { "label": "elemento", "properties": { } } }	"grafoManual"	8	34	52

Started streaming 1 records after 28 ms and completed after 87 ms.

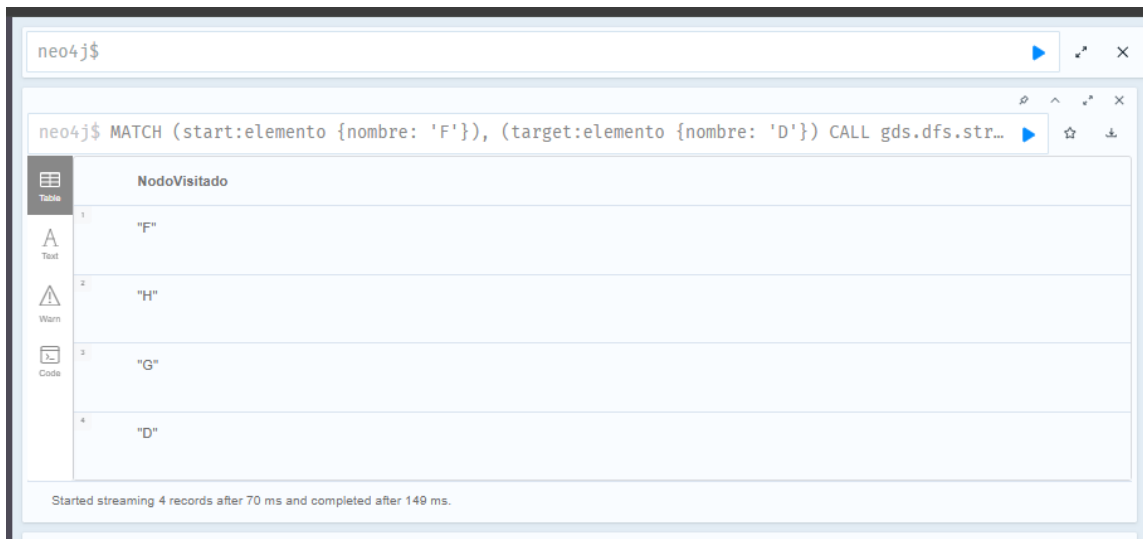
Se ha ejecutado el algoritmo BFS partiendo del nodo **H**.

- **Resultado:** El algoritmo visita los nodos por niveles (primero los vecinos directos de H, luego los vecinos de estos).
- **¿Se recorren todos los nodos?: Sí.** Al ser un grafo **conexo** (todos los nodos están enlazados entre sí directa o indirectamente), el recorrido en anchura acaba alcanzando todos los nodos del sistema.

The screenshot displays the Neo4j Browser interface. The top section shows a query execution for a Breadth-First Search (BFS) starting from node 'H'. The query is: `MATCH (start:elemento {nombre: 'H'}) CALL gds.bfs.stream('grafoManual', { sourceNode: ...`. The results are shown in a table with one column, 'nombre', containing the following values: 'H', 'A', 'B', 'D', 'F', and 'G'. Below the table, a status message indicates: 'Started streaming 8 records after 38 ms and completed after 72 ms.'

The bottom section shows a second query execution: `LOAD CSV WITH HEADERS FROM 'https://raw.githubusercontent.com/mathbeveridge/asoiaf/refs/he...`. The results are displayed in a table with two columns: 'Server version' (Neo4j/4.4.15) and 'Server address' (localhost:7687). The 'Query' column contains the full query text. The 'Summary' column shows the query execution details, including the number of nodes created (796) and the number of properties set (1592). The 'Response' column shows the result of the query execution.

c) Recorrido en Profundidad (DFS) de F a D



The screenshot shows a Neo4j query interface with the command: `neo4j$ MATCH (start:elemento {nombre: 'F'}), (target:elemento {nombre: 'D'}) CALL gds dfs.str...`. The results are displayed in a table titled 'NodoVisitado' with 4 rows. The first column is an index (1-4) and the second column is the node name ('F', 'H', 'G', 'D'). A status bar at the bottom indicates: 'Started streaming 4 records after 70 ms and completed after 149 ms.'

	NodoVisitado
1	"F"
2	"H"
3	"G"
4	"D"

Se ha realizado el recorrido empezando en **F** con el objetivo de llegar a **D**.

- **Resultado:** A diferencia del BFS, el DFS explora "hacia el fondo" de una rama antes de retroceder.
- **¿Se recorren todos los nodos?:** No necesariamente se recorren todos los nodos. El algoritmo se detiene en el momento en que encuentra el nodo objetivo (**D**), dejando sin visitar aquellas ramas que no formaban parte de la ruta explorada hasta ese instante

d) Camino mínimo (Dijkstra) entre D y F

Sin distancia (Menor número de saltos)



The screenshot shows a Neo4j query interface with the command: `neo4j$ MATCH (source:elemento {nombre: 'D'}), (target:elemento {nombre: 'F'}) CALL gds.shorteste...`. The results are displayed in a table titled 'Camino' with 2 columns: 'Camino' and 'Saltos'. There is one row showing the path ['D', 'H', 'F'] and the number of hops (2.0). A status bar at the bottom indicates: 'Started streaming 1 records after 44 ms and completed after 155 ms.'

	Camino	Saltos
1	["D", "H", "F"]	2.0

Con distancia (Usando propiedad "distancia")

The screenshot shows the Neo4j Desktop interface with two queries executed.

Query 1: `neo4j$ MATCH (source:elemento {nombre: 'F'}), (target:elemento {nombre: 'D'}) CALL gds.shortestPath('grafoManual', source, target, {relationshipProperties: 'distancia'})`

Result 1:

Camino	DistanciaTotal
["F", "G", "D"]	7.0

Started streaming 1 records after 44 ms and completed after 88 ms.

Query 2: `neo4j$ CALL gds.graph.project('grafoManual', 'elemento', 'camino', { relationshipProperties: 'distancia' })`

Result 2:

nodeProjection	relationshipProjection	graphName	nodeCount	relationshipCount	projectMillis
<pre>{ "elemento": { "label": "elemento", "properties": { "distancia": { "defaultValue": null, "property": "distancia", "aggregation": "DEFAULT" } } } }</pre>	<pre>{ "camino": { "orientation": "NATURAL", "indexInverse": false, "aggregation": "DEFAULT", "type": "camino", "properties": { "distancia": { "defaultValue": null, "property": "distancia", "aggregation": "DEFAULT" } } } }</pre>	"grafoManual"	8	4	152

Started streaming 1 records after 28 ms and completed after 196 ms.

Query 3: `$ MATCH (n) DETACH DELETE n; CREATE (a:elemento {nombre: 'A'}), (b:elemento {nombre: 'B'}), (c:elemento {nombre: 'C'})`

Se han realizado dos pruebas del algoritmo de Dijkstra para comparar el efecto de los pesos:

1. **Sin tener en cuenta la propiedad "distancia":** El algoritmo trata todas las aristas con un peso uniforme (1). El resultado es el camino con menos saltos: ["D", "H", "F"] (Coste: 2.0).
2. **Teniendo en cuenta la propiedad "distancia":** Aquí el algoritmo suma los valores de la propiedad distancia. El camino óptimo cambia a: ["F", "G", "D"] con una **Distancia Total de 7.0**.

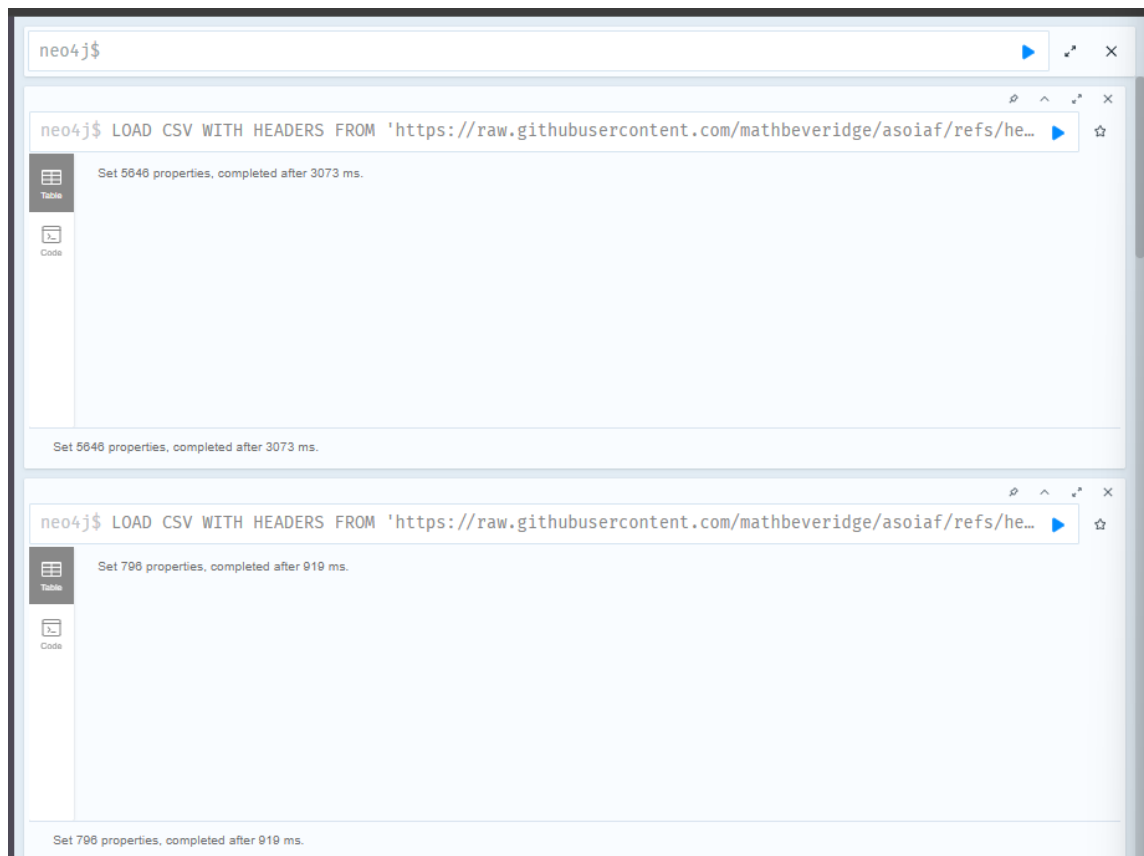
Ejercicio 2. Medidas de centralidad en los siete reinos

Vamos a estudiar diversas relaciones entre los personajes de Juego de Tronos. Las siguientes relaciones se han obtenido evaluando cuando dos personajes coinciden de diversas maneras:

a) y b) Carga de datos (Dataset Juego de Tronos)

Se han importado los datos de la saga "Canción de Hielo y Fuego" desde los repositorios indicados:

- **Nodos:** Se han creado los nodos con la etiqueta: **Personaje**, asignando las propiedades **id** y **nombre** desde el CSV de nodos.
- **Aristas:** Se han creado relaciones: Interacciones de forma **bidireccional**. Se han procesado las propiedades **peso** (**weight**) como *float* y **libro** (**book**) como *integer*, utilizando la función *coalesce* para gestionar posibles valores nulos.



C) Análisis de Relevancia (Centralidad)

neo4j\$

neo4j\$ CALL gds.graph.project('gotGraph', 'Personaje', 'Interacciones', {relationshipProperti...

Table

Text

Code

	nodeProjection	relationshipProjection	graphName	nodeCount	relationshipCount	projectMillis
1	{ "Personaje": { "label": "Personaje", "properties": { } } } }	{ "Interacciones": { "orientation": "NATURAL", "indexInverse": false, "aggregation": "DEFAULT", "type": "Interacciones", "properties": { "peso": { "defaultValue": null, "property": "peso", "aggregation": "DEFAULT" } } } }	"gotGraph"	796	2823	102

Started streaming 1 records after 56 ms and completed after 185 ms.

neo4j\$ CALL gds.graph.drop('gotGraph', false)

Table

Text

Code

	graphName	database	memoryUsage	sizeInBytes	nodeCount	relationshipCount	configuration	density	creation
1	"gotGraph"	"neo4j"	"	-1	796	2823	{ } }	0.004460984166113587	"2020-09-29T15:00:00"

neo4j\$

neo4j\$ CALL gds.degree.stream('gotGraph') YIELD nodeId, score RETURN gds.util.asNode(nodeId)...

Table

Text

Code

	Personaje	Grado
1	"Cersei Lannister"	80.0
2	"Arya Stark"	80.0
3	"Catelyn Stark"	66.0
4	"Jaime Lannister"	66.0
5	"Daenerys Targaryen"	58.0

Started streaming 5 records after 24 ms and completed after 69 ms.

neo4j\$

```
neo4j$ CALL gds.betweenness.stream('gotGraph') YIELD nodeId, score RETURN gds.util.asNode(nod...
```

	Personaje	Intermediacion
1	"Jon Snow"	11817.17839890448
2	"Eddard Stark"	6696.248374009368
3	"Jaime Lannister"	6680.473811221545
4	"Robb Stark"	5972.671070109796
5	"Daenerys Targaryen"	5959.627095252417

Started streaming 5 records after 22 ms and completed after 245 ms.

neo4j\$

```
neo4j$ CALL gds.beta.closeness.stream('gotGraph') YIELD nodeId, score RETURN gds.util.asNode(...
```

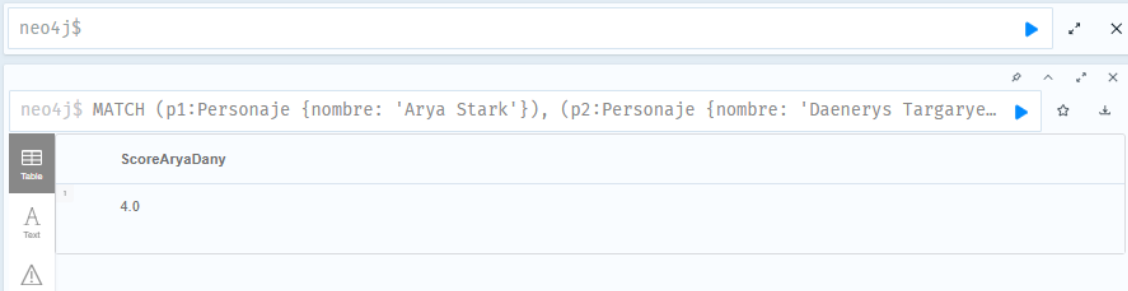
	Personaje	Cercania
1	"Arthur Dayne"	1.0
2	"Arianne Martell"	1.0
3	"Aemon Targaryen (Maester Aemon)"	1.0
4	"Arthor Karstark"	1.0
5	"Arya Stark"	1.0

Started streaming 5 records after 21 ms and completed after 64 ms.

Tras proyectar el grafo (gotGraph), se identificaron los personajes clave según tres métricas:

- **Grado (Degree):** Identifica a los más conectados. Destacan **Cersei Lannister** y **Arya Stark** (80.0 conexiones), siendo los núcleos de comunicación más activos.
- **Cercanía (Closeness):** Mide la rapidez de acceso al resto de la red. Personajes como **Arthur Dayne** presentan valores máximos (1.0), indicando una posición estratégica en sus respectivas comunidades.
- **Intermediación (Betweenness):** Identifica "puentes" entre grupos. **Jon Snow** lidera ampliamente esta métrica, siendo vital para conectar comunidades que de otro modo estarían aisladas narrativamente.

d) Predicción de enlaces

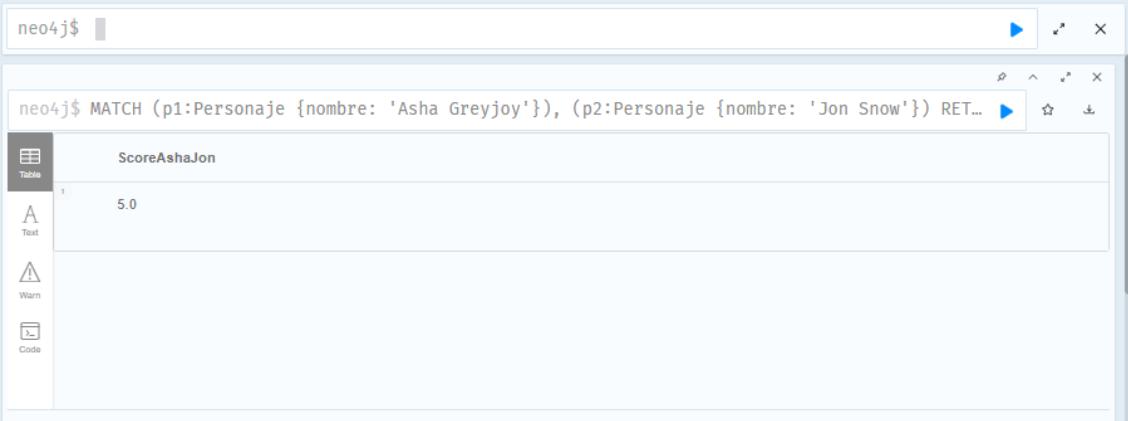


neo4j\$

```
neo4j$ MATCH (p1:Personaje {nombre: 'Arya Stark'}), (p2:Personaje {nombre: 'Daenerys Targarye...
```

ScoreAryaDany	
1	4.0

Table
Text
Warn



neo4j\$

```
neo4j$ MATCH (p1:Personaje {nombre: 'Asha Greyjoy'}), (p2:Personaje {nombre: 'Jon Snow'}) RET...
```

ScoreAshaJon	
1	5.0

Table
Text
Warn
Code

Started streaming 1 records after 30 ms and completed after 38 ms.

Se han comparado las probabilidades de interacción futura entre dos pares de personajes utilizando los métodos de predicción de enlaces (Common Neighbors, Adamic Adar, etc.):

- **Arya Stark y Daenerys Targaryen:** Obtienen una puntuación de **4.0**.
- **Asha Greyjoy y Jon Snow:** Obtienen una puntuación de **5.0**.

Comentario: Existe una mayor probabilidad estadística de que se produzca un nuevo contacto entre **Asha Greyjoy y Jon Snow**, dado que su puntuación en los algoritmos de predicción de enlaces es superior a la del par Arya-Daenerys.

Conclusión Final

El análisis realizado demuestra la versatilidad de las bases de datos orientadas a grafos para extraer conocimiento tanto de estructuras abstractas como de redes sociales complejas. Mientras que el Ejercicio 1 permite validar la lógica fundamental de los algoritmos de búsqueda y optimización de rutas (Dijkstra), el Ejercicio 2 revela la estructura subyacente de una narrativa masiva.

Las métricas de centralidad confirman que la relevancia de un personaje no solo depende de cuánta gente conoce (Grado), sino de su capacidad para actuar como nexo de unión entre diferentes facciones (Intermediación), posición que ocupa Jon Snow. Asimismo, las técnicas de Predicción de Enlaces permiten anticipar desarrollos narrativos basados en la densidad de la red, demostrando que Neo4j es una herramienta predictiva potente más allá del simple almacenamiento de datos

